



Automated Compliance for Embedded C.

Embedded C Coding Guidelines

**Practical Coding Rules for Reliable, Maintainable,
and Safety-Conscious Embedded C**

Beta Edition • Version 0.9 • August 2026

ComplyC Project

About This Guide

The ComplyC Embedded C Coding Guidelines provide a practical coding baseline for C developers, with particular emphasis on embedded and safety-conscious software. The guide is designed to be used alongside the ComplyC static-analysis tool during development, review, and beta evaluation.

- Readability and maintainability
- Predictable control flow and bounded complexity
- Defect prevention and review consistency
- Static analyzability
- Safety-conscious C practices

ComplyC does not replace formal standards such as MISRA C, CERT C, ISO 26262, AUTOSAR guidance, or an organization's engineering process. A clean ComplyC report must not be represented as formal compliance with any third-party standard.

Guideline Classification

Classification	Meaning
Required	Expected for the default ComplyC coding profile.
Recommended	Strong engineering practice; may require review or optional configuration.
Advisory	Useful depending on project architecture and constraints.

1. Naming Conventions

Function Names

NAMING_FUNC_001

Required: Use descriptive lower_snake_case function names.

Preferred

```
uint16_t calculate_crc(void);  
void update_motor_state(void);  
bool is_voltage_valid(void);
```

Avoid

```
uint16_t CalculateCRC(void);  
void updateMotorState(void);  
void func1(void);
```

Global Variables

NAMING_GLOBAL_001

Required: Global variables shall use the configured global prefix, typically g_.

Preferred

```
uint16_t g_system_voltage;  
bool g_system_initialized;
```

File-Static Variables

NAMING_STATIC_001

Required: File-local static variables shall use the configured static prefix, typically s_.

Preferred

```
static uint16_t s_retry_count;  
static bool s_initialized;
```

Macro Names

NAMING_MACRO_001

Required: Macro names shall use UPPER_SNAKE_CASE.

Preferred

```
#define MAX_RETRY_COUNT 3U  
#define ADC_TIMEOUT_MS 100U
```

Avoid

```
#define maxRetryCount 3
#define Timeout 100
```

Variable Names

NAMING_VAR_004

Recommended: Use names that communicate purpose; single-letter names are normally limited to conventional short-lived loop counters.

Preferred

```
uint16_t retry_count;
uint32_t timeout_ms;
uint16_t battery_voltage_mv;
```

Avoid

```
uint16_t x;
uint32_t val;
uint16_t temp;
```

Identifier Length

NAMING_VAR_003

Required: Keep identifiers within the configured project limit. The beta profile uses 31 characters.

Include Units in Names

Engineering review

Recommended: Include physical units in identifiers when this prevents ambiguity.

Preferred

```
uint32_t timeout_ms;
uint16_t voltage_mv;
uint32_t frequency_hz;
```

2. Source Documentation

File Header

FILE_HEADER_001

Required: Each source file should contain the configured module documentation header.

Preferred

```
/******  
 * Module Name: VoltageMonitor  
 * Description: Monitors supply voltage and reports diagnostic faults.  
 * Author: ComplyC Project  
 * Version: 1.0  
******/
```

Comments Explain Intent

Engineering review

Recommended: Comments should explain engineering intent, assumptions, constraints, or non-obvious decisions.

Preferred

```
/* Require three consecutive failures before reporting the diagnostic. */  
failure_count++;
```

Avoid

```
/* Increment counter */  
failure_count++;
```

3. Formatting

Mandatory Braces

FORMAT_BRACE_001

Required: Use braces for control-flow bodies.

Preferred

```
if (system_ready)
{
    start_control();
}
```

Avoid

```
if (system_ready)
    start_control();
```

Brace Placement

BRACE_STYLE_002

Required: The default beta profile uses an opening brace on a new line.

Preferred

```
if (condition)
{
    process_data();
}
else
{
    report_error();
}
```

One Statement Per Line

Engineering review

Recommended: Keep one logical statement per line to improve reviewability.

4. Function Design

Function Length

FUNC_SIZE_001	Required: Keep functions focused. The default beta profile limits function length to 40 lines.
----------------------	--

Parameter Count

FUNC_PARAMS_001	Required: The default beta profile limits functions to 6 parameters.
------------------------	--

Cyclomatic Complexity

FUNC_CC_001	Required: The default beta profile limits cyclomatic complexity to 10.
--------------------	--

Nesting Depth

FUNC_NESTING_001	Required: The default beta profile limits control-flow nesting to 4 levels.
-------------------------	---

Empty Functions

DEAD_CODE_EMPTY_FUNC_001	Required: Do not leave unintentionally empty functions. Document intentional interface stubs.
---------------------------------	---

5. Control Flow

Assignment in Conditions

**MISRA_LITE_ASSIGN_C
OND_001**

Required: Do not place assignments inside conditional expressions.

Preferred

```
status = read_status();

if (STATUS_VALID == status)
{
    process_status();
}
```

Avoid

```
if (status = read_status())
{
    process_status();
}
```

Complete If/Else-If Chains

CTRL_ELSEIF_001

Required: The default profile expects an if/else-if chain to terminate with an else branch.

Switch Default

**CTRL_SWITCH_DEFAULT
T_001**

Required: Every switch statement shall contain a default label.

Preferred

```
switch (state)
{
    case STATE_IDLE:
        handle_idle();
        break;

    default:
        report_invalid_state();
        break;
}
```

Switch Fall-Through

CTRL_SWITCH_FALLTHROUGH_001

Required: Terminate switch cases explicitly unless fall-through is intentional and documented.

Empty Statements

DEAD_CODE_EMPTY_STMT_001

Required: Avoid accidental empty statements such as a semicolon immediately after an if condition.

Preferred

```
if (error_detected)
{
    shutdown_system();
}
```

Avoid

```
if (error_detected);
{
    shutdown_system();
}
```

Empty Blocks

CTRL_EMPTY_BLOCK_001

Required: Document intentionally empty control blocks.

Unreachable Code

DEAD_CODE_UNREACHABLE_001

Required: Remove statements that cannot execute after unconditional control transfer.

6. Safety-Conscious Embedded C

Dynamic Memory

SAFETY_DYNAMIC_MEMORY_001

Required: The default embedded profile forbids malloc, calloc, realloc, and free. Prefer bounded static allocation or an approved deterministic memory strategy.

Recursion

SAFETY_RECURSION_001

Required: The default embedded profile forbids recursive call chains because stack consumption can become difficult to bound.

Infinite Loops

LOOP_INFINITE_001

Required: Uncontrolled infinite-loop patterns are forbidden by the default profile. Legitimate scheduler/main loops require project-specific handling or documented deviation.

goto

FORBIDDEN_GOTO_001

Required: The default profile forbids goto. Prefer structured control flow and focused helper functions.

Selected Unsafe APIs

SAFETY_FORBIDDEN_API_001

Required: The default profile flags selected APIs such as gets, strcpy, strncpy, strcat, strncat, sprintf, vsprintf, scanf, sscanf, and fscanf. Use project-approved bounded interfaces.

7. Additional Engineering Guidance

Macros

Engineering review

Recommended: Prefer typed constants, enums, static inline functions, or ordinary functions when a macro is not necessary. Parenthesize macro parameters and expressions, and avoid repeated-evaluation side effects.

Preferred

```
#define ADD(a, b) ((a) + (b))
```

Avoid

```
#define ADD(a, b) a + b
```

Variable Initialization

Engineering review

Recommended: Initialize objects before use. Deeper initialization analysis may require data-flow analysis.

Magic Numbers

magic_number handler / profile dependent

Recommended: Replace unexplained literals with named constants where they represent domain thresholds, limits, or configuration values.

Preferred

```
#define MAX_OPERATING_TEMP_DEG_C 125

if (temperature > MAX_OPERATING_TEMP_DEG_C)
{
    shutdown();
}
```

Avoid

```
if (temperature > 125)
{
    shutdown();
}
```

Header Guards

Engineering review

Recommended: Protect header files against multiple inclusion and minimize unnecessary dependencies.

Preferred

```
#ifndef VOLTAGE_MONITOR_H
#define VOLTAGE_MONITOR_H

void voltage_monitor_init(void);

#endif
```

Defensive Embedded C

Engineering review

Recommended: Validate external inputs, check buffer boundaries, use fixed-width types where appropriate, bound execution, handle invalid states, and check failure-return paths.

8. Worked Example

Problematic Implementation

```
int ProcessData(int x)
{
    char buffer[10];

    if (x = 1)
        strcpy(buffer, "START");

    switch (x)
    {
        case 1:
            start_system();
    }

    return 0;

    log_result();
}
```

- Non-conforming function naming
- Assignment inside a condition
- Missing braces
- Selected unsafe API
- Missing switch default
- Unreachable code

Improved Implementation

```
int process_data(int input_state)
{
    int result = STATUS_OK;

    if (STATE_START == input_state)
    {
        result = process_start_state();
    }
    else
    {
        result = STATUS_INVALID_STATE;
    }

    return result;
}
```

9. ComplyC Beta Quick Reference

Rule ID	Purpose
NAMING_FUNC_001	Function naming
NAMING_GLOBAL_001	Global naming
NAMING_STATIC_001	Static naming
NAMING_MACRO_001	Macro naming
NAMING_VAR_003	Identifier length
NAMING_VAR_004	Single-letter identifiers
FILE_HEADER_001	File documentation
FORMAT_BRACE_001	Mandatory braces
BRACE_STYLE_002	Brace placement
FUNC_SIZE_001	Function length
FUNC_CC_001	Cyclomatic complexity
FUNC_NESTING_001	Nesting depth
FUNC_PARAMS_001	Parameter count
DEAD_CODE_EMPTY_FUNC_001	Empty functions
MISRA_LITE_ASSIGN_COND_001	Assignment in conditions
CTRL_ELSEIF_001	Complete conditional chains
CTRL_SWITCH_DEFAULT_001	Switch default
CTRL_SWITCH_FALLTHROUGH_001	Switch fall-through
DEAD_CODE_EMPTY_STMT_001	Empty statements
CTRL_EMPTY_BLOCK_001	Empty control blocks
DEAD_CODE_UNREACHABLE_001	Unreachable code
SAFETY_RECURSION_001	Recursion
SAFETY_DYNAMIC_MEM_001	Dynamic memory
LOOP_INFINITE_001	Infinite loops
FORBIDDEN_GOTO_001	goto
SAFETY_FORBIDDEN_API_001	Selected unsafe C APIs

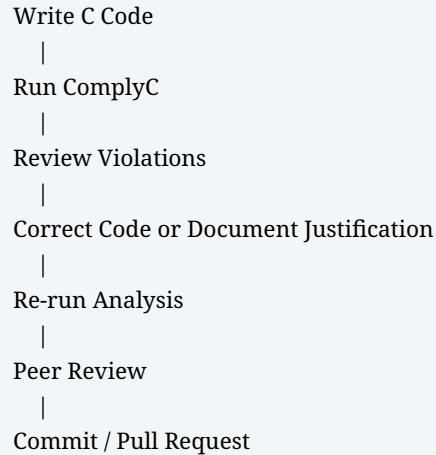
Automated Checks vs. Engineering Review

ComplyC beta automates a defined set of naming, formatting, complexity, control-flow, dead-code, and safety-oriented checks. Engineering judgment is still required for architecture, abstraction quality, comment usefulness, concurrency behavior, hardware correctness, requirements compliance, and justified deviations.

Relationship to Industry Standards

These guidelines reflect established C and embedded-software practices and may overlap conceptually with MISRA C, CERT C, ISO 26262-oriented engineering practice, and other coding guidance. ComplyC is not MISRA-certified, and successful analysis does not establish MISRA, CERT, ISO 26262, ASPICE, cybersecurity, or regulatory compliance.

Recommended Workflow



```
graph TD; A[Write C Code] --> B[Run ComplyC]; B --> C[Review Violations]; C --> D[Correct Code or Document Justification]; D --> E[Re-run Analysis]; E --> F[Peer Review]; F --> G[Commit / Pull Request];
```

Write C Code
|
Run ComplyC
|
Review Violations
|
Correct Code or Document Justification
|
Re-run Analysis
|
Peer Review
|
Commit / Pull Request

References

1. Carnegie Mellon University-hosted C Coding Standard:
<https://users.ece.cmu.edu/~eno/coding/CCodingStandard.html>
2. ISO/IEC 9899 - Programming Languages - C.
3. MISRA C - Guidelines for the Use of the C Language in Critical Systems.
4. SEI CERT C Coding Standard.
5. ComplyC beta rule engine and default coding-style configuration.

License and Attribution

This guide is part of the ComplyC Project. It presents original ComplyC guidance and descriptions of the ComplyC rule set while drawing on established software-engineering principles and publicly available C coding guidance. Third-party standards, names, and trademarks remain the property of their respective owners.